Lab Procedure
Servomotor

Introduction

Ensure the following:

1. You have reviewed the Application Guide – Servo
2. Make sure you have the provided **Servomotor**.
3. Make sure the **Mechatronic Actuators Trainer** is out of its case; it is powered using the provided power supply and connected to the PC.
    a. Turn it on using the Power switch on the side of the device.
    b. Both the USB and Power LED should be green.

This experiment consists of two parts that will help you understand how a servomotor works and how to drive it. First, you will explore how Pulse Width Modulation (PWM) works and the average signal value it represents. The second section of the lab will walk you through setting up bias and incremental values to move a servo to various positions.

Pulse Width Modulation (PWM)

In this section, we will explore the workings of PWM using a visualization that represents the average brightness of a white LED.

1. Launch Visual Studio Code or your preferred Python IDE. If using Visual Studio Code, click on **File** > **Open Folder**..., and browse to the directory containing the python files and lab procedure for the servomotor lab (.../actuators_trainer/1_fundamentals/servo/hardware/python). Open this directory.
2. Open pwm_exploration.py and run the script using the button or the terminal. This is set to run for a total duration of 5 minutes (300 seconds), which can be adjusted by setting the **`totalTime`** parameter at the top of the script to a different number.
3. A window will pop up (PWM Visualization, shown in Figure 1) which lets you interactively observe the mean value of a modulated signal, each part of the window is described below,
    A. This zone will continuously alternate between white (pixel value 255) and black (pixel value 0). The amount of time spent in either black or white depends on the current **duty cycle** (shown in your python terminal, for example `Duty Cycle is at 50 %.`), which represents the ON (white) duration within the modulated pulse. The duty cycle is capped between 3 and 96%.

B. This zone represents the average value of the signal during each period. For example, for a 30% duty cycle, the average value between ON (white or 255) and OFF (black or 0) would be

$$0.30 \times 255 \ + \ 0.70 \times 0 \ = \ 76$$

Or a gray pixel value of 76 (from a 0-255 range).

C. This zone represents a slider allowing you to adjust the duty cycle using your **left and right arrow keys** to decrease or increase the value respectively.

The image at the bottom is static and represents the information above.

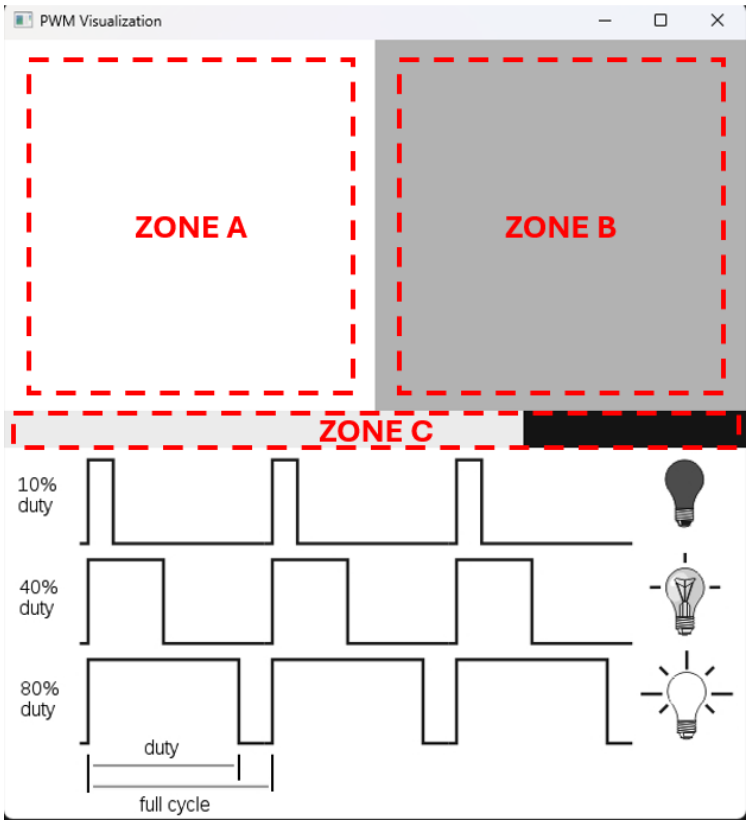

Figure 1. PWM Visualization

4. Stop the script by using **`Ctrl+C`** in the terminal and restart the application. Note that the **duty cycle** slider will be at 50% upon restarting. Comment on the ON/OFF durations from part A and gray color intensity from part B.

5. Use the right arrow key and increase the **duty cycle** in the 75% to 85% range. Comment on the ON/OFF durations from part A and gray color intensity from part B.

6. Use the left arrow key and decrease the **duty cycle** in the 15% to 25% range. Comment on the ON/OFF durations from part A and gray color intensity from part B.

7. Stop the script using **`Ctrl+C`**.

In real life scenarios, the frequency of the pulse is much faster. As such, you cannot visualize the individual ON or OFF states (for example, part A of the visualization). This is how dimmable LEDs brightness is controlled, and no flickering is observed by the naked eye.

Driving a Servomotor

Pulse Width Modulation in the previous section was demonstrated using an example to control the brightness of an LED. In such a case, the duty cycle varied between 0% (always OFF, black and no brightness) and 100% (always ON, white and fully bright). Servomotors, on the other hand, are driven using PWM with smaller ranges of possible duty cycles. There are two universally accepted ranges for servomotor duty cycles.

In the first scenario, a 20-millisecond period PWM signal (50Hz) is used with the duty cycle varying between 1 to 2 milliseconds. This range of duty cycle takes the servo from one extreme to another.

In the second scenario, a 20-millisecond period PWM signal (50Hz) is used with the duty cycle varying between 0.5 to 2.5 milliseconds. This range of duty cycle takes the servo from one extreme to another.

Warning: It is important to identify which range of duty cycle your servo represents. If using a 1 to 2 millisecond pulse to drive a servo that expects 0.5 to 2.5 milliseconds, the servo's effective range will be cut in half. However, using a 0.5 to 2.5 millisecond pulse to drive a servo that expects 1 to 2 milliseconds may result in the servo trying to exceed its limits, potentially breaking the servomotor.

8. Open servomotor.py and locate the following line:

   ```
   with ActuatorsTrainer(block = 2, servoMode = 0) as actuators:
   ```

   This line is present in all labs and is used to initialize the device. The parameter **`block = 2`** dictates what block, as printed on the device, you are going to use. You can update this parameter if you connect the motors to a different block. The **`servoMode = 0`** indicates the range of the PWM signal (0 for 0.5 to 2.5ms, 1 for 1 to 2ms).

9. Connect your motor to the block you will be using.
   a. Use the **`Servo`** connection for the motor power input. Check the connection polarity (brown/red/yellow connected to -/+/S).

10. Open the datasheet for the provided motor located in the servo folder (or the datasheet for your own servomotor). Find the information related to the duty cycle range. For the provided motor's datasheet, the information is under the *5. Control Specification* section, particularly pulse width range. Note that the provided values are in μs and not ms. On the line to initialize the device in servomotor.py, set the **`servoMode`** parameter to 0 or 1 to match the corresponding servo pulse width range.

    The Actuators Trainer motor API simplifies the servo command option for users to a range of 0 to 1. Based on the **`servoMode`** selected in the **`ActuatorsTrainer`** initialization command, the firmware on the trainer will automatically map the 0 to 1

range of command to the required millisecond value for the servo you specify. If **`servoMode`** is 0, the 0-1 command will map to 0.5 to 2.5ms, but if **`servoMode`** is 1, the 0-1 command will map to 1 to 2ms. In both cases, a command of 0.5 would map to 1.5 ms command since it's the halfway point of both ranges.

11. From the datasheet, find information related to motor deadband, paying attention to the units of the provided value. On the script, after the initialization of the device, find the variable **`delta = 0`** and set the **`delta`** parameter to the minimum incremental command that will move the motor based on the datasheet information. Note that the commanded value will be a minimum incremental command based on the 0 to 1 range. Run servomotor.py. Use the UP and DOWN arrow keys in your keyboard to move the servo intermittently. Does the servo respond consistently? If not, stop your code, adjust the **`delta`** parameter and run the code again. Note down the **`delta`** parameter.

12. Bring the **`servoCmd`** to 0 and then to 1. If your servo has a servo arm attached, look at where the arm is at when you are at 0 or 1. Once it is at one of the limits (command of 0 or 1), place the servo arm perpendicular to the servo, so the arm is pointing out of the servo as shown in the image.

13. Stop and re-run servomotor.py. Note that the initial **`servoCmd`** is set to 0.5. Is the servo arm at the 0 position on the servomotor? Based on how you placed the arm on the previous step, is the arm now pointed up (or down) since it is in the middle point? If it is not, use the UP and DOWN arrow keys to bring the servo arm to the middle position (90˚ from the position in the picture) and note the Servo Command printed in the terminal. Stop the script using **`Ctrl+C`** and set an appropriate value to the **`bias`** variable, such that a 0.5 command brings the load in the middle 0 position. Note down your **`bias`** value.

14. Rerun the servomotor.py script. With the appropriate **`delta`** and **`bias`** parameters, use the UP and DOWN arrow keys on your keyboard to drive the servo the extreme positions. Note the maximum and minimum servo commands you can produce using the keyboard, as well as the value at which the servo stops moving. The **`servoCmd`** values between 0 and 1 should be able to drive the complete range of motion for the servo (±90 degrees in each direction if using the provided DMOND DCS16LP servo) as shown in Figure 2. Did the bias term affect the limits of the motor?

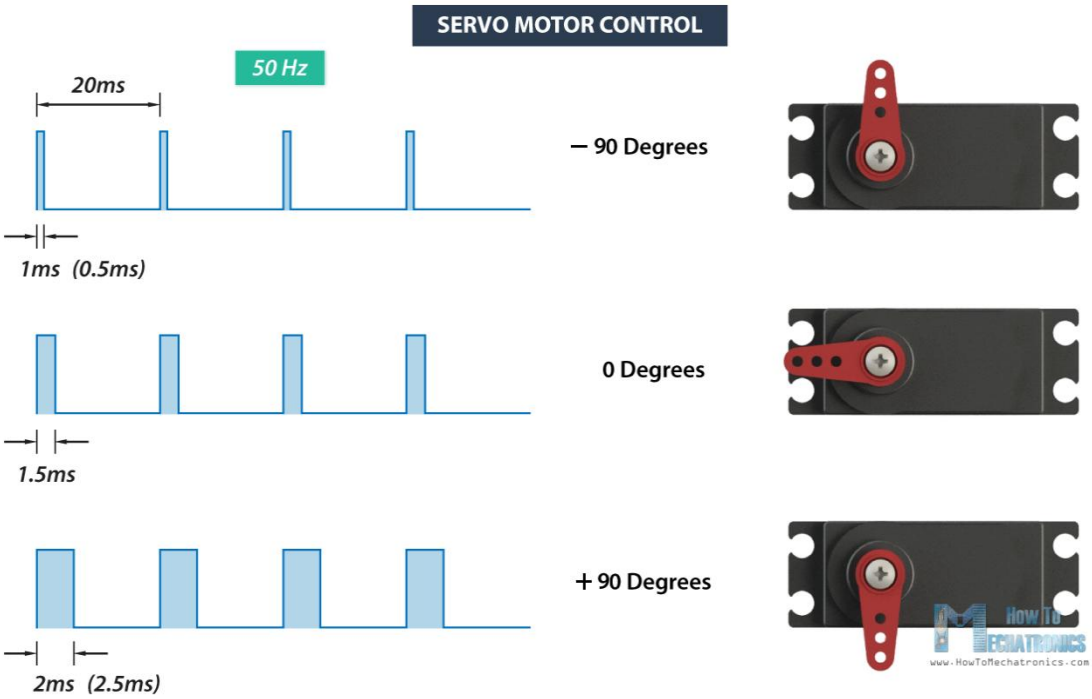

Figure 2. Servomotor PWM

15. Stop the script using **`Ctrl+C`** and uncomment the lines that are commented out in the keyboard state section they both start with **`# if servoCmd`**. What do these code lines do? Rerun the script and verify that the servomotor can reach it's maximum and minimum positions.

16. Stop your script.

17. Optional (only try this step if you are using the provided DMOND DCS16LP servo or a servo that accepts 0.5 to 2.5 ms commands). Change the **`servoMode`** parameter to 1 on the initialization for the board. Rerun the script. What do you observe? Why is this?

18. Stop, save and close your program.

19. Power OFF the Mechatronic Actuators Trainer, disconnect any motors and pack them along the power supply and USB cables in the provided case.